



US Cyber Games Beginner's Game Room 2025

Drive Discovery Official Writeup

Prepared by Asa Reynolds, Abby O'Connell, Ardian Peach, and Bailey Dalton

For this challenge, we're given part of a disk image, a file representation of an actual hard drive and its contents. To open disk images, we can use a well developed tool called Autopsy to analyze and look around inside for any files (it can even catch deleted ones!). Autopsy is also commonly used in law enforcement for digital forensics purposes. You can download it here:

<https://www.autopsy.com/download/>

Using Autopsy

Once you open Autopsy, you will need to create a new case, which is essentially a project folder where you can import disk files to analyze. The details you put in here such as the case name and number don't matter, just make sure you save the case itself in a folder where you can easily keep track of it. Once you've made the case, a window should pop up prompting you to add a new data source, if it doesn't, you can click the green plus button in the top left to do so. We can now select the 001 file we've downloaded to load into Autopsy!

Steps

1. **Select Host**
2. Select Data Source Type
3. Select Data Source
4. Configure Ingest
5. Add Data Source

Select Host

Hosts are used to organize data sources and other data.

☒ Generate new host name based on data source name

☐ Specify new host name

Select Data Source Type



Disk Image or VM File

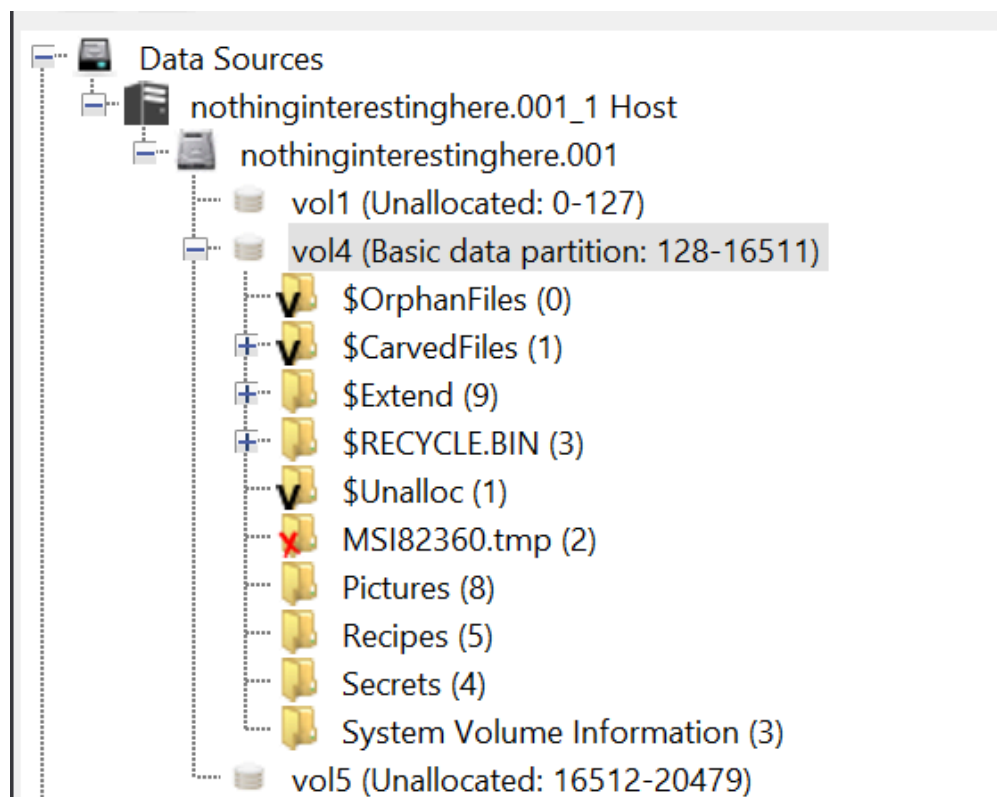
Select Data Source

Path:

C:\Users\HackUCF\Downloads\nothinginterestinghere.001

Browse

On the “Configure Ingest” page you can just leave all the default checks on. Click next, and you should see the nothinginterestinghere file loaded in. You will need to click into it and expand the sub-trees.



We can now see some folders from the operating system and whoever was using this disk before. Let's take a look at some secrets, shall we?

Base64 Encoding & Decoding

Interesting. We can see two .txt files, one that's been deleted (as indicated by the red x, though Autopsy picked it up) and one that's a note to self. Deleted files can often be of interest in forensic CTF challenges. In the note, this person mentions a *really secure type of encryption called Base64*. Looking at the flag.txt, there does appear to be obfuscated text.

Name	S	C	O	Modified Time	Change Time	Access Time	Created T
[current folder]				2025-05-05 10:28:10 EDT	2025-05-05 10:28:10 EDT	2025-05-05 10:28:10 EDT	2025-05-05 10:28:10 EDT
[parent folder]				2025-05-05 10:27:36 EDT	2025-05-05 10:27:36 EDT	2025-05-05 10:27:36 EDT	2025-05-05 10:27:36 EDT
flag.txt				2025-05-05 10:26:36 EDT	2025-05-05 10:26:36 EDT	2025-05-05 10:26:36 EDT	2025-05-05 10:26:36 EDT
note to self.txt			2	2025-05-05 10:26:12 EDT	2025-05-05 10:26:12 EDT	2025-05-05 10:26:12 EDT	2025-05-05 10:26:12 EDT

Data Content

HexTextApplicationFile MetadataOS AccountData ArtifactsAnalysis ResultsContextAnnotationsOther Occurrences

StringsExtracted TextTranslation

Page: 1 of 1PageGo to Page:

NOTES:
1. Make sure to delete flag.txt before giving this USB drive to anyone.
2. Apparently there's a really secure type of encryption called Base64, I should look into using that.

flag.txt				2025-05-05 10:26:36 EDT	2025-05-05 10:26:36 EDT	2025-05-05 10:26:36 EDT	2025-05-05 10:26:36 EDT
note to self.txt			2	2025-05-05 10:26:12 EDT	2025-05-05 10:26:12 EDT	2025-05-05 10:26:12 EDT	2025-05-05 10:26:12 EDT

Data Content

HexTextApplicationFile MetadataOS AccountData ArtifactsAnalysis ResultsContextAnnotationsOther Occurrences

StringsExtracted TextTranslation

Page: 1 of 1PageMatches on page: - of - Match100%Reset

U1ZCUkd7ZDNsMzczZF9uMDdfZjByNjA3NzNuXzI4MzAyOTM4Mn0=

Base64 is a form of **encoding**, not encryption. It's a binary-to-text encoding scheme that lets you represent any sequence of bytes using only 64 printable ASCII characters, so the data can travel safely through channels that accept text but mangle or reject raw binary. It's especially useful when trying to send a bunch of data or text that could be interpreted incorrectly by a program (like a payload with a bunch of spaces and other special characters). By Base64 encoding that, you're just turning it into a bunch of simple ASCII characters, so that the new

simple text form can travel around as just regular ASCII characters, and then it can be decoded later into its original form, special characters and all.

Base64 can be effortlessly transcribed back into its original form with a simple online tool. An extremely popular tool used by cybersecurity specialists and CTF players is **CyberChef**, an all in one tool that can handle all sorts of encryption, decryption, and formatting of all sorts of things. Seriously, it can do just about anything!

<https://gchq.github.io/CyberChef/>

CyberChef allows you to make a recipe by dragging and dropping various operations from the left side panel. For this, all we're going to need is the "From Base64" operation, so we'll drag that to the middle. If you wanted to convert something into Base64 (say, for making a CTF challenge), you could use the "To Base64" operation. Then, we'll paste the encoded string we found in flag.txt into the Input panel on the right side and ... voila!

The screenshot shows the CyberChef web interface. On the left, the 'Operations' panel lists various tools, with 'From Base64' highlighted. The 'Recipe' panel in the center contains a single operation: 'From Base64'. Below this operation, there is a dropdown menu set to 'Alphabet' with the range 'A-Za-z0-9+/' and a checked checkbox for 'Remove non-alphabet chars'. The 'Input' panel on the right contains a long Base64-encoded string: 'U1ZCUkd7ZDNsMzczZF9uMDdFZjByNjA3NzNuXzI4MzAyOTM4Mn0='. The 'Output' panel at the bottom right displays the decoded result: 'SVBRG{d3l373d_n07_f0r60773n_283029382}'.

Challenge solved!

Flag: `SVBRG{d3l373d_n07_f0r60773n_283029382}`